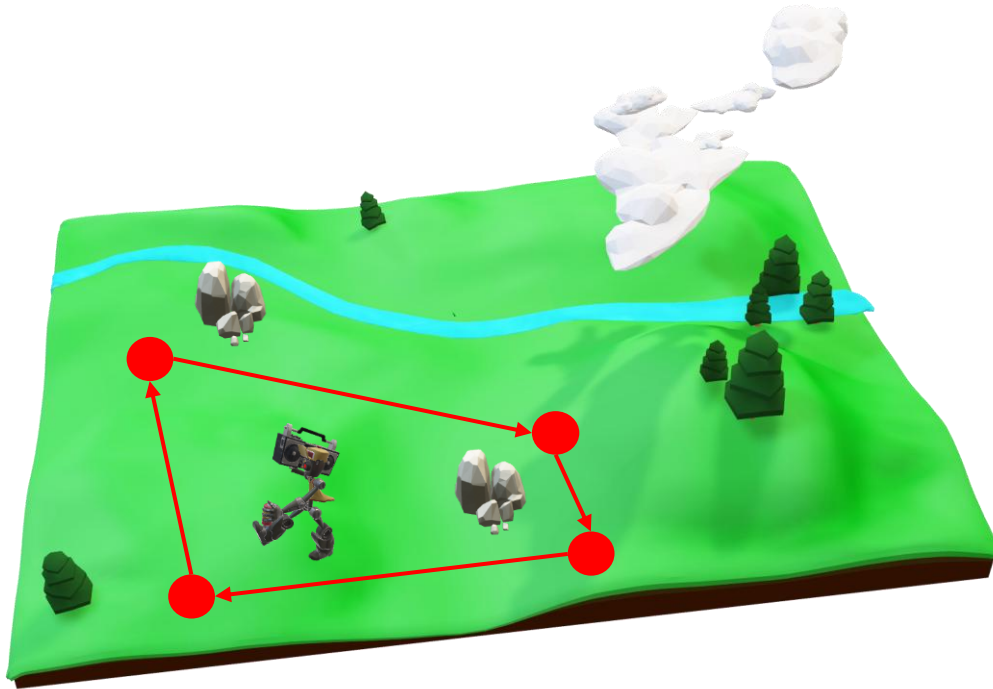


NPCの行動制御③(巡回)

NPCは、配置して停止しているままだと、不具合のように見えます。
かといって、ネズミのようにランダムに移動されると、
意思を持っていない動きになりまし、ゲームのバランスも崩れてしまいます。

多くのNPCは、「巡回ルート」を持っていて、
制作者の意図通りに、ゲームで動作するように作られています。



ゲーム開発では、これを「経路(パス)」といいます。
最もシンプルな実装方法としては、ウェイポイント法というのがあり、
上図のように巡回ポイント(座標)をいくつか用意して、順番に移動していく
やり方です。

この実装だけだと、移動途中で障害物などがあった場合、
障害物に引っかかってしまって、それ以上移動できなくなります。

そういった問題を解消するため、座標間の移動には、
経路探索(ダイクストラやAstarやNavMesh)が使用され、
目標地点に移動するための最適ルートを検索するアルゴリズムを使用します。

一旦、今回は経路探索は行わず、
障害物に引っかからないような巡回ポイントを作って、
ウェイポイント法による、巡回行動を制作していきます。

```
EnemyRobot.h
```

```
class EnemyRobot : public EnemyBase
{

public:

    ~ 省略 ~

    // 描画
    void Draw(void) override;

private:

    ~ 省略 ~

    // 巡回ルート座標
    std::vector<VECTOR> wayPoints_;

    // 現在、移動先としている巡回先ルートのインデックス
    int activeWayPointIndex_;

    // 現在、移動先としている巡回先座標
    VECTOR nextWayPoint_;

    ~ 省略 ~

    // 巡回ルートの移動方向を設定する
    void SetMoveDirPatrol(void);
```

```
EnemyRobot.cpp
```

```
void EnemyRobot::Draw(void)
{

    // 基底クラスの描画処理
    CharactorBase::Draw();

#ifdef _DEBUG
```

```

// 巡回ルート描画
for (const auto& point : waypoints_)
{
    DrawSphere3D(
        point, 50.0f, 10,
        0x0000ff, 0x0000ff, false);
}

#endif // _DEBUG

}

void EnemyRobot::InitPost(void)
{
    ~ 省略 ~

    // 巡回ルート
    waypoints_.emplace_back(VGet(1926.18f, 1.76f, 618.34f));
    waypoints_.emplace_back(VGet(2553.30f, -11.82f, -593.32f));
    waypoints_.emplace_back(VGet(1400.34f, -26.21f, -457.11f));
    waypoints_.emplace_back(VGet(1274.85f, -49.35f, 168.02f));

    // 初期状態設定
    ChangeState(STATE::THINK);
}

void EnemyRobot::ChangeStateThink(void)
{
    stateUpdate_ = std::bind(&EnemyRobot::UpdateThink, this);

    // 思考
    // ランダムに次の行動を決定
    // 30%で待機、70%で徘徊
    int rand = GetRand(100);
    if (rand < 30)
    {
        ChangeState(STATE::IDLE);
    }
}

```

```

else
{
    ChangeState (STATE::PATROL);
}

}

void EnemyRobot::ChangeStatePatrol (void)
{

    stateUpdate_ = std::bind (&EnemyRobot::UpdatePatrol, this);

    // 移動量ゼロ
    movePow_ = AsoUtility::VECTOR_ZERO;

    if (activeWayPointIndex_ == waypoints_.size())
    {
        // 巡回終了
        ○○○
        ChangeState (STATE::THINK);
        return;
    }

    // 次の巡回ポイント更新
    ○○○

    // 巡回ルート of 移動方向を設定する
    SetMoveDirPatrol ();

    // 移動スピード
    moveSpeed_ = 5.0f;

    // 歩きアニメーション再生
    animationController_>Play(
        static_cast<int>(ANIM_TYPE::WALK), true);

}

void EnemyRobot::UpdatePatrol (void)
{

```

```

// 巡回ポイントとの球体衝突判定(半径30.0fくらい)
○○○
{
    // 巡回ポイントインデックス更新
    ○○○

    // 次の移動地点へか、待機か思考
    ChangeState(STATE::THINK);
    return;
}

// 巡回ルートの移動方向を設定する
SetMoveDirPatrol();

// 移動量の計算
○○○

}

void EnemyRobot::SetMoveDirPatrol(void)
{

    // 巡回先座標XZ
    VECTOR tmpPos = nextWayPoint_;
    tmpPos.y = 0.0f;

    // 現在地座標XZ
    VECTOR pos = transform_.pos;
    pos.y = 0.0f;

    // XZ平面上の移動方向を計算
    moveDir_ = VNorm(VSub(tmpPos, pos));

}

```

【要件】

ロボットが巡回ポイントを辿って移動するよう、
プログラムを完成させること。

- ・ 見づらいなので、移動可能範囲のデバッグ描画をコメント

【目標】

ロボットが巡回すること。



【チャレンジ】

余裕があったら、巡回ポイントも外部ファイル化してみましょう。